

Kappa Language — Getting Started

Adi Dasgupta
Student No. 103552167

November 30, 2025

Purpose

This document is a short, high-level submission README for the Kappa project. It contains:

- A few **tested working examples** and their outputs
- A few **tested failing examples** (type / parse errors) to demonstrate that the type checker rejects incorrect programs
- Notes and current limitations (so the grader does not need implementation internals)

How to run

You can run the interpreter in two main ways:

- Run a file:

```
python3 kappa.py run examples/demo.kappa
```

- Start the REPL:

```
python3 kappa.py repl
```

Note: On some systems you may need `python` instead of `python3`.

Tested, working examples

These examples were executed on the provided interpreter; the exact output (captured during tests) is shown.

1. Basic arithmetic

```
# code
1 + 2 * 3

# observed output
7
```

2. Variables and functions

```
# code
let x = 10
let inc = fn y: Int -> y + 1
inc x

# observed output
11
```

3. Factorial (recursion via fix)

```
# code (self-contained)
((fix (fn self: (Int -> Int) -> fn n: Int ->
  if n == 0 then 1 else n * (self (n - 1))
)) 5)

# observed output
120
```

4. Fibonacci (recursion via fix)

```
# code (self-contained)
((fix (fn self: (Int -> Int) -> fn n: Int ->
  if n == 0 then 0
  else if n == 1 then 1
  else (self (n - 1)) + (self (n - 2))
)) 6)

# observed output
8
```

Tested examples that produce type errors (intentionally)

The handout requires also demonstrating some incorrect programs that the type system must reject. Below are examples and the interpreter output that demonstrates rejection.

1. If branches with different types

```
# code
if true then 1 else false

# observed output
Error: Type error: Branches of if must have same type
```

2. Adding a Boolean to an Integer

```
# code
1 + true

# observed output
Error: Type error: Function expects TInt() got TBool()
```

3. Applying a non-function

```
# code
let x = 5
x 10

# observed output
Error: Type error: Call of non-function: got TInt()
```

4. Heterogeneous list literal

```
# code
[1, true]

# observed output
Error: Type error: List items must have the same type
```

Notes & Limitations

- A few language constructs were slightly fragile during automated tests:
 - * Some list pattern-match syntaxes and tuple syntaxes were sensitive to exact spacing/parentheses in the input. These are listed in the tutorial and annotated.
 - * In some test runs some helper builtins (e.g. `print` or `fst/snd`) required specific invocation forms; the tutorial shows the safe form to use.
- The interpreter prints helpful error diagnostics including where the parser/type-checker failed; in automated tests the error text was captured and is shown above.
- If desired, we can make the interpreter return a non-zero exit code on error to make CI/grading stricter; currently (for some tests) error messages appear on stdout while the process exit code was 0 — this is easy to change if required by the course.

Where to find more

- Full runnable examples and exercise files are in the `examples/` directory.
- Solutions are in `examples/solutions/` (for instructor reference).
- A longer tutorial (with more runnable examples) is provided in `TUTORIAL.md` and included in the separate Tutorial PDF.